

PERSONALIZED ALGORITHMIC TRADING PLATFORM USING DEEP LEARNING AND REINFORCEMENT LEARNING

MAJOR PROJECT REPORT

*Submitted in partial fulfilment of the requirements for
the award of degree*

OF
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted By:

Rhythm Jain (22103126) , Pratham Garg (22103121)
Raghav Gupta (22106085) , Padala Balaji (22103115))

Under the supervision of:

Dr. Samayveer Singh
Assistant Professor



Department of Computer Science and Engineering
Dr. B R Ambedkar National Institute of Technology
Jalandhar - 144008 (Punjab) India
June, 2026

CANDIDATES DECLARATION

We hereby certify that the work presented in this project report entitled **“PERSONALIZED ALGORITHMIC TRADING PLATFORM USING DEEP LEARNING AND REINFORCEMENT LEARNING”** in partial fulfillment of the requirement for the award of a Bachelor of Technology degree in Computer Science and Engineering, submitted to the Dr. B R Ambedkar National Institute of Technology, Jalandhar is an authentic record of our own work carried out during the period from July 2026 to May 2025 under the supervision of Dr. Samayveer Singh, Assistant Professor, Department of Computer Science & Engineering, Dr. B R Ambedkar National Institute of Technology, Jalandhar. We have not submitted the matter presented in this report to any other university or institute for the award of any degree or any other purpose.

Rhythm Jain (22103126)
Pratham Garg (22103121)
Raghav Gupta (22106085)
Padala Balaji (22103115)

This is to certify that the above declaration made by the candidate is correct and true to the best of my knowledge.

Dr. Samayveer Singh
Assistant Professor
Dept. of CSE

Dr. A L Sangal
Head and Professor
Dept. of CSE

Supervisor

HOD, CSE

ACKNOWLEDGEMENT

It is true that hundreds of people work behind the scenes for the success of a play. The end result of the project required a lot of guidance and help from many people and our group was very lucky to receive this during the course of the project. Whatever we are today is only due to such supervision and assistance and we thank them from the bottom of our hearts.

We would like to express our deepest gratitude to our project mentor Dr. Samayveer Singh, Assistant Professor, who believed in our ideas and suggested new ideas when needed. He fully supported us in solving our problems. We would like to express our deepest gratitude to Prof. A L Sangal, Head of the Department of Computer Science and Engineering, for his direct and indirect support. We are grateful to Dr Aruna Malik, Coordinator Major Project for providing us mentors and all other support.

We are extremely thankful to have constant encouragement and guidance from all the Faculties of the Department of Computer Science & Engineering. We would also like to express our sincere thanks to all laboratory staff for their timely support.

Rhythm Jain (22103126)
Pratham Garg (22103121)
Raghav Gupta (22106085)
Padala Balaji (22103115)

ABSTRACT

Algorithmic trading has evolved from deterministic, rule-based engines into adaptive, datadriven systems capable of learning from dynamic market conditions. Despite substantial progress in predictive modeling and decision optimization, most deployed platforms remain disconnected in their design—prediction modules operate independently of execution engines, and neither is sensitive to the individual risk preferences of the end user. This report presents a personalized AI-assisted algorithmic trading platform that unifies deep learning, reinforcement learning, and behavioral profiling within a single, production-oriented architecture, designed specifically for Indian equity markets (NSE and BSE). The system is implemented using Python 3.12, FastAPI, PyTorch, Stable-Baselines3 for Proximal Policy Optimization (PPO), OpenAI Gymnasium, Next.js 16, React 19, PostgreSQL 15, and Docker Compose. The backend architecture is organized into three functional layers. Layer 1 handles market data acquisition through yfinance for 52 NIFTY 50 constituent equities, computes over 30 technical indicators via pandas-ta, and constructs a 30-dimensional state representation for the reinforcement learning agent. Layer 2 constitutes the decision engine, wherein a custom Gymnasium trading environment accommodates a five-action discrete action space (HOLDBUY, HOLDSELL, BUY, SELL, IDLE) and a 34-dimensional observation space. A PPO agent with a [256, 256] multilayer perceptron policy is trained for 30,000 timesteps using the Sharpe ratio as the primary reward signal. Layer 3 implements trader behavior modeling through a 30-question psychometric assessment, generating a 15-dimensional behavior vector that governs position sizing via the Kelly Criterion, risk categorization, and user-specific model retraining. A two-layer Long Short-Term Memory (LSTM) model with 64 hidden units and approximately 39,000 trainable parameters is employed for next-day closing price prediction, achieving a best validation Mean Squared Error (MSE) of 0.000180. The PPO agent achieves a mean return of 132.28 percent across five evaluation episodes with a Sharpe ratio of 0.66. The platform exposes functionality through 15 REST endpoints and a WebSocket interface, and operates in three deployment modes: in-memory demonstration, Firebasebacked cloud deployment, and full production mode with PostgreSQL and Redis. A resilient fallback mechanism ensures continued operation in the absence of trained model artifacts, returning rule-based signals without system interruption.

PLAGIARISM CHECK

We have checked plagiarism for our Project Report for our project a Turnitin. We are thankful to our mentor Dr. Samayveer Singh for guiding us at this. Below is the digital receipt. Plagiarism is approximately 6%.

Document Details

Submission ID
trn:oid::1:3578665852

Submission Date
May 25, 2026, 5:36 PM GMT+5:30

Download Date
May 25, 2026, 5:38 PM GMT+5:30

File Name
VPN_Project_plg.pdf

File Size
439.0 KB

26 Pages

5,877 Words

35,499 Characters

 Page 1 of 30 - Cover Page Submission ID trn:oid::1:3578665852

Figure 1: Page 1 of plagiarism report

 Page 2 of 30 - Integrity Overview Submission ID trn:oid::1:3578665852

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **27 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

-  **6% Internet sources**
-  **2% Publications**
-  **3% Submitted works (Student Papers)**

Integrity Flags

0 Integrity Flags for Review
No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Figure 2: Page 2 of plagiarism report

Contents

Certificate	i
Acknowledgement	ii
Abstract	iii
Plagiarism Check	iv
List of Figures	viii
List of Tables	ix
Chapter 1 Introduction	1
1.1 Indian Market Characteristics	2
1.2 Motivation for the Proposed Work	2
1.3 Problem Statement	3
1.4 Objectives	3
1.5 Feasibility Analysis	4
1.5.1 Technical Feasibility	4
1.5.2 Operational Feasibility	4
1.5.3 Feasibility Study: Economic	5
Chapter 2 Literature Review	6
2.1 Recurrent Neural Networks for Financial Time Series Forecasting	6
2.1.1 Long Short-Term Memory Networks	6
2.1.2 BiLSTM and Attention-based Approach	7
2.1.3 Probabilistic Forecasting using DeepAR	7
2.2 Reinforcement Learning for Trading	7
2.2.1 Frameworks of RL Applied to Trading	7
2.2.2 Proximal Policy Optimization in Finance	8

2.2.3	Trading Environment Customizations	8
2.3	Systematic Reviews of Deep Learning in Trading Algorithms	8
2.4	Optimal Position Sizing Based on the Kelly Criterion and Risk Management in Algorithmic Trading	9
2.4.1	Table of Summary Comparison	9
Chapter 3	Proposed Methodology	11
3.1	Technical Foundations	11
3.1.1	Long Short-Term Memory Networks	11
(a)	Architecture and Gating Mechanisms	11
(b)	Proposed LSTM Architecture	13
3.1.2	Reinforcement Learning Framework	14
(a)	Formulation of the Problem as a Markov Decision Process	14
(b)	Proximal Policy Optimization	14
(c)	Reward Function	15
3.1.3	Kelly Criterion for Position Sizing	16
(a)	Mathematical Formulation	16
(b)	Practical Implementation Variations	16
3.1.4	Sharpe Ratio	17
3.2	System Architecture	18
3.2.1	Layer 1: Data Processing	19
3.2.2	Layer 2: Decision Engine	20
3.2.3	Layer 3: Trader Behavior and Risk Profiling	21
3.2.4	Application and API Layer	22
3.3	Deployment Modes and Resilience	23
Chapter 4	Experiment Results	25
4.1	Experimental Setup	25
4.1.1	Data Acquisition and Preprocessing	25
4.1.2	LSTM Training Configuration	26
4.1.3	PPO Training Configuration	26
4.2	LSTM Performance Evaluation	26
4.2.1	Quantitative Results	26

4.2.2	Interpretation	27
4.3	Evaluation of PPO Agent Performance	28
4.3.1	Performance in terms of Return and Risk-Adjusted Metrics	28
4.3.2	Results Interpretation	28
(a)	Sharpe Ratio Importance	28
(b)	Return Value Importance	29
(c)	Training Performance	29
4.4	System Performance	29
4.4.1	Signal Generation Latency	29
4.4.2	WebSocket Data Pushing	30
4.4.3	Trader Behavior Profiler	30
4.5	Reliability and Scalability Discussion	31
4.5.1	Fallback Mechanism	31
4.5.2	Multi-Stock Scalability	32
4.6	Limitations of Evaluation	32
4.6.1	Simulation Fidelity	33
4.6.2	Overfitting Risk	33
4.6.3	Hardcoded PPO Confidence	33
4.6.4	State Dimension Padding	33
4.6.5	Absence of Walk-Forward Validation	34
Chapter 5 Conclusion		35
5.1	Conclusion	35
5.2	Future Work	36

List of Figures

Fig. 1	Page 1 of plagiarism report	iv
Fig. 2	Page 2 of plagiarism report	iv
Fig. 3.1	High-level architecture of the personalized algorithmic trading platform.	19
Fig. 3.2	LSTM Training and Validation	20

List of Tables

Table 2.1	Comparative evaluation of existing works in deep learning and reinforcement learning in algorithmic trading	10
Table 3.1	Deployment modes and infrastructure requirements.	23
Table 4.1	Dataset characteristics for LSTM training.	25
Table 4.2	PPO agent training configuration.	26
Table 4.3	LSTM model training and evaluation metrics.	27
Table 4.4	PPO agent performance metrics across five test episodes.	28

CHAPTER 1

INTRODUCTION

Financial markets are among the most complex dynamic systems considered within modern science. Prices at each point in time encapsulate the collective response of millions of market participants to macroeconomic data, political news, institutional orders, and psychological factors specific to individuals. Such complexity poses intrinsic challenges to human traders: information processing limits, interference of emotions, latency in responses, and inconsistency in following trading strategies. Algorithmic trading was designed partially to overcome such constraints by translating the logic of trading into computational procedures that can make quick, accurate, and impartial decisions. Simple rule-based systems that appeared in the 1970s and 1980s relied on deterministic procedures, like moving averages, momentum indicators, and arbitrage opportunities. In addition to faster executions and consistency in applying rules, such approaches have several critical drawbacks. First, a system tailored to a trending regime performs poorly under mean-reversion. Conversely, a strategy optimized to detect mean-reversion does not work under trending conditions. Moreover, the lack of flexibility is the most severe restriction of the approach: rule-based systems do not learn from experience and cannot adapt to changes in distribution. The introduction of machine learning and later deep learning has significantly revolutionized the landscape of algorithmic trading. Neural networks based on recurrent architecture, especially Long Short-Term Memory networks, have a clear advantage in modeling financial time series and capturing time dependencies that can be ignored by shallow networks. At the same time, the Reinforcement Learning approach to building an algorithm is conceptually distinct: contrary to learning a static mapping between inputs and outputs, a policy for choosing actions is learned by interacting with the environment. In a trading scenario, reinforcement learning naturally fits the process of sequential decision making motivated by rewards. Proximal Policy Optimization algorithm, introduced by Schulman et al. [1] has been chosen for application in finance precisely because of the stability of training

dynamics. Indeed, the clipped surrogate loss function guarantees that the policy updates will not be overly aggressive and thus may cause instability. An LSTM model predicting prices combined with reinforcement learning driven by PPO and adjusted according to the behavioral characteristics of individual traders constitutes the primary motivation behind the project.

1.1 Indian Market Characteristics

National Stock Exchange of India and Bombay Stock Exchange have several features which distinguish them from their US counterparts researched extensively before. Equity trading at NSE occurs between 09:15 to 15:30 Indian Standard Time on normal trading days with a preopen trading window between 09:00 to 09:15. The NIFTY 50 stock market index used as benchmark for trading activity here includes 52 underlying stocks belonging to different sectors such as information technology, banking, energy, consumer goods, and pharmaceuticals—note the intentional overstatement of number of shares in index methodology. Indian equity markets feature a higher retail participation compared to developed markets and exhibit different dynamics as a result. High participation by retail traders in market activity will cause prices to move even further during quarterly reporting periods, national budgets releases, and macroeconomic data released by Reserve Bank of India. Trading system implementation has to account for intraday circuit breakers, rules regulating high-frequency trading in accordance to SEBI guidelines, and tick-by-tick data policy of stock exchanges. Market-specific features require the implementation of any Indian equities trading platform to include proper normalization of symbols (.NS suffix for NSE and .BO for BSE at Yahoo Finance) and attention to trading hours as well as liquidity conditions of specific stocks.

1.2 Motivation for the Proposed Work

Even with numerous contributions in academia on the topic of applying deep learning and reinforcement learning to trading, there still remains considerable disparity between theoretical research and real-world trading applications. More specifically, three areas of improvement can be highlighted. Integration Gap. Almost all literature either covers forecasting or decision making in isolation. While price forecast models generally do not feature the ability to convert prices into actionable trading decisions within some sort of risk framework, RL-based trading agents are inclined to use an observation

vector instead of incorporating price predictions produced by specific prediction models. The current project combines those elements into one pipeline. **Personalization Gap.** In conventional automated trading frameworks, there are no options to customize the trading strategy to an individual user’s risk preference because of the assumption that investors are homogeneous in their behavior. In other words, a conservative individual managing his or her retirement assets and a speculative trader working with discretionary funds have completely different optimization targets. Thus, the current platform incorporates a personalized behavior vector of 15 dimensions generated using psychological questionnaires. **Deployment Gap.** Trading algorithms developed for the sake of research are generally simulated in isolation; few papers mention deployment in production environments. In this particular project, three levels of system integration were implemented: in-memory deployment for demo purposes, Firebase-backed deployment on a cloud, and deployment in the production environment utilizing a combination of PostgreSQL and Redis as well as fallback strategies in the absence of trained model artifacts.

1.3 Problem Statement

This paper seeks to provide a solution to the following challenges related to the development of an end-to-end algo-trading platform that meets all the following criteria at once: (i) Collecting and processing market data for 52 stocks listed on the NSE/BSE market with the creation of a set of indicators using real-time data; (ii) Creating trade signals with the use of LSTM-based next-day price forecasting and PPO- based policy optimization; (iii) Customizing trade signals and position size according to a particular user’s behavioral pattern obtained from a structured questionnaire; (iv) Disclosing platform functionality through REST API and WebSocket APIs which will be accessed through a full-featured web interface; and (v) Based on the resilience of the platform, ensuring its stable functioning in the conditions of insufficient infrastructure with the lack of trained models. In summary, the criteria defined above represent the link between machine learning algorithms and algo-trading technology.

1.4 Objectives

The following technical goals will be achieved in this project:

- (1) Development of a pipeline for historical OHLCV data ingestion and normalization

for 52 NIFTY 50 stocks with calculation of over 30 technical indicators. (2) Development of a two-layer LSTM model to predict next-day closing prices in Indian stocks dataset. (3) Implementation of custom Gymnasium environment with five discrete actions and reward mechanism based on Sharpe ratio. (4) Model training and evaluation process for PPO agent using Stable-Baselines3. (5) Development of a psychometric behavioral profiling system producing 15-dimensional behavior vector mapped to risk categories. (6) Kelly Criterion-based position sizing adjusted to individual risk profiles. (7) Development of a FastAPI-based web server offering REST and WebSocket APIs with three deploy modes and failure recovery behavior. (8) Implementation of a Next.js 16 frontend web application with visualizations and behavioral profiling screens.

1.5 Feasibility Analysis

1.5.1 Technical Feasibility

All software packages used by the application are mature, well-maintained, and have extensive documentation. FastAPI supplies a robust, high-performing backend framework along with auto-generated documentation. PyTorch supplies a framework to perform all required machine learning tasks, including training and prediction using LSTMs. Stable-Baselines3 is an industry-standard PPO framework featuring extensive configuration options. Gymnasium provides standardized interfaces for reinforcement learning environments. Next.js v16 and React 19 are used for the front-end development, providing server-side rendering capabilities. Computation needs for training are low as the LSTM model takes just under five minutes to go through 30 training epochs for 23,167 samples on a CPU. Training of PPO on 30,000 steps takes from two to five minutes. Both algorithms have relatively small numbers of parameters. The LSTM network contains approximately 39,000 parameters, and the fully connected policy network in PPO has two hidden layers with 256 neurons each.

1.5.2 Operational Feasibility

The demonstration deployment is purely memory-based without depending on any external database, thereby facilitating quick assessment in environments where resources are limited. The production deployment utilizes Docker Compose to manage a total of four microservices: Fast API back end, Next JS front-end, PostgreSQL 15, and Redis 7. All configuration settings use environment variables with reasonable default values.

1.5.3 Feasibility Study: Economic

The whole software architecture is open-source, making any licensing fees negligible to nil. Data from 52 stocks of the NIFTY 50 index are gathered using yfinance, which offers free access to data from Yahoo Finance and does not require subscription. Both development and inference tasks are carried out on an ordinary laptop machine with 8 GB RAM. The estimated monthly cost of running cloud services for such a solution ranges from 6to25. In case of opting for market feed subscription for real-time operations, the fee would vary between 25to125 per month.

/sectionStructure of the Report The rest of this report proceeds as follows. The literature on the subject under investigation, including previous research in deep learning-based price forecasting, reinforcement learning-based trading, and hybrid approaches is reviewed in Chapter 2. The suggested system, including its architecture, layer-by-layer functionality and implementation is described in Chapter 3. The experimental approach used, along with the results obtained, including LSTM training results, PPO evaluation measures, and overall system efficiency characteristics, are given in Chapter 4. Chapter 5 provides economic considerations of the proposed platform. Chapter 6 concludes this report and outlines directions for future research. Chapter 7 contains the bibliography of all works referred to in this report.

CHAPTER 2

LITERATURE REVIEW

There has been tremendous progress made in the area of algorithmic trading via machine learning techniques within the last ten years, and the literature on this topic can be broadly categorized into three approaches: supervised learning for prediction purposes, reinforcement learning for decision making, and the third which involves a combination of both. This chapter is devoted to analyzing some of the seminal papers in each of these approaches.

2.1 Recurrent Neural Networks for Financial Time Series Forecasting

2.1.1 Long Short-Term Memory Networks

In what is considered a groundbreaking work in neural network research, Hochreiter and Schmidhuber [2] proposed the Long Short-Term Memory (LSTM) framework as a solution to the problem of vanishing gradients which prevented conventional recurrent neural networks (RNNs) from learning long-term dependencies. Through the use of gating mechanisms, the cell state of an LSTM network can be selectively allowed to persist or change over arbitrarily long intervals. This trait makes LSTMs particularly appealing in the field of finance because of the rich autocorrelation structures, seasonality, and conditional memory in stock prices. Specifically, Fischer and Krauss [3] utilized LSTM networks for predicting the S&P 500 index components, producing statistically significant returns in comparison with other models including random forest and deep neural networks. The proposed LSTM network model comprised a single LSTM layer of 25 hidden units trained on 15-year daily returns of 500 equities, illustrating the capabilities of recurrent neural networks in forecasting the highly volatile asset prices. However, the study did not focus on position size, risk assessment, and trading rules formulation.

2.1.2 BiLSTM and Attention-based Approach

A novel deep reinforcement learning approach incorporating BiLSTM with multi-head attention for algorithmic trading of DJIA components was introduced by Huang et al. [4]. The key idea is the use of the bidirectional model which enables each step to pay attention to information from the past and the future in the training window, something that cannot be done using the unidirectional version. Attention weightings provide a way to highlight important temporal information. The results obtained using DJIA stock data showed improved performance relative to both the buy-and-hold benchmark strategy and multiple competing RL baselines. However, this research ignored the issue of trader heterogeneities and personal behavior.

2.1.3 Probabilistic Forecasting using DeepAR

A DeepAR-Attention based forecast model has been used in [5], which expanded the use of LSTM-based prediction to that of probabilistic forecasting. Instead of predicting a point estimation result, it predicts the parameters of the probability distribution, thereby estimating the uncertainty involved in forecasting and enabling direct application to position-sizing based on risks. The experiments have been conducted in Chinese equity markets, and therefore the research can be applied to emerging markets, which would be comparable to India's equity market. While the proposed platform takes its inspiration from such uncertainty modeling, it uses a more basic approach with LSTM.

2.2 Reinforcement Learning for Trading

2.2.1 Frameworks of RL Applied to Trading

A pioneer effort on applying RL algorithms to trading was introduced by Moody and Saffell [6] where the portfolio selection problem was modeled as MDP and recurrent neural network models were trained based on a policy gradient formulation. They showed that optimizing financial utility instead of prediction error could lead to better results. The computational limitations of that time limited the scope of experimentation. Later on, the popular deep Q-network (DQN) architecture proposed by Mnih et al. [7] for Atari gaming environments was applied to trading scenarios by many researchers. While the strength of DQN in estimating Q-values from high dimensional states motivated its application to financial tasks, its off-policy formulation and unstable estimation of Q-values in non-

stationary finance domain proved to be difficult.

2.2.2 Proximal Policy Optimization in Finance

Proximal Policy Optimization by Schulman et al. [1] is an on-policy algorithm designed to mitigate the training instabilities associated with previous policy gradient techniques by implementing a surrogate objective function with a clipping term. This term restricts the ratio of probabilities between old and new policies to be within the range of $[1 - \epsilon, 1 + \epsilon]$, ensuring that large updates are not made to parameters, all while retaining sample efficiency comparable to trust region approaches. Such features make PPO extremely useful in finance settings characterized by noisy and non-stationary reward signals. PPO has been proven effective in multiple studies related to trading applications. In fact, Stable-Baselines3, which offers a state-of-the-art production-ready implementation of PPO [8], has become a go-to choice among RL traders, enabling the creation of reproducible experiments with well-known conventions in hyperparameters tuning. This study utilizes Stable-Baselines3's implementation of PPO with [256, 256] MLP policy and learning rate of 3×10^{-4} , using a reward function based on Sharpe ratio.

2.2.3 Trading Environment Customizations

The FinRL library [9] is an approach towards standardization of RL trading research through creation of a universal set of Gymnasium-like environments for trading on stock, option, and cryptocurrencies exchanges. FinRL allows comparing various RL algorithms such as DQN, PPO, A2C, DDPG, and SAC using reference implementation and identical market setting. The proposed platform follows the Gymnasium environment design idea used in FinRL but introduces a customized TradingEnv suitable for trading on the Indian stock exchange environment.

2.3 Systematic Reviews of Deep Learning in Trading Algorithms

A systematic review by Bhuiyan et al. [10] examined more than 100 publications on the application of deep learning to algorithmic trading involving recurrent neural nets, convolutional nets, transformer-based models, and hybrid approaches. This paper pointed to some common patterns within deep learning literature related to algorithmic trading: Deep learning models outperform their shallow counterparts when it comes to modeling complex relationships in the financial markets; however, issues with

overfitting, explainability, and deployment stability remain open research questions. Of these open questions, the need for personalized trading algorithms is addressed here.

2.4 Optimal Position Sizing Based on the Kelly Criterion and Risk Management in Algorithmic Trading

Based on Kelly’s criterion [11], which stems from information theory and gambling, there exists an optimal method of sizing positions that can optimize the long-term growth rate of capital. This concept has been extensively researched in the context of financial trading. There have been adaptations of the optimal formula due to its high sensitivity to estimates of the probability of win and pay-off ratio [12], which include, among others, fractional Kelly and adjusted Kelly based on volatility. In the current system, the Kelly criterion is used to determine position size with a cap at percent.

2.4.1 Table of Summary Comparison

The following table provides a systematic comparison of select representative literature with respect to methodology, dataset, performance, limitations, and the research gaps addressed by the proposed study.

Table 2.1: Comparative evaluation of existing works in deep learning and reinforcement learning in algorithmic trading

Author(s) & Year	Method	Dataset/Market	Strengths	Limitations & Research Gaps
Fischer & Krauss (2018)	LSTM	S&P 500, 15 years daily	Demonstrates superiority over RF for equity return prediction	No RL, no position sizing, no personalization
Huang et al. (2024)	BiLSTM-Attention + RL	DJIA daily	Strong temporal pattern extraction; attention interpretability	US market only; no behavioral profiling; no web deployment
Li et al. (2024)	DeepAR-Attention	Chinese equities	Probabilistic forecasting; uncertainty quantification	No RL decision making; no trader personalization
Bhuiyan et al. (2025)	Systematic Review	100+ papers	Comprehensive landscape analysis	Identifies gap in integrated prediction-RL-personalization systems
Liu et al. (2021)	FinRL (DQN, PPO, A2C)	US equities, crypto	Standardized RL environment; multi-algorithm support	No behavioral profiling; no web interface; US-centric
Moody & Saffell (2001)	Policy Gradient RNN	Forex	Foundational RL trading framework	Computationally constrained; no deep learning
Mnih et al. (2015)	DQN	Atari (applied to trading)	Pioneered deep RL value functions	Q-value instability in non-stationary markets
Schulman et al. (2017)	PPO	MuJoCo, Atari	Stable on-policy training; clipped objective	General RL algorithm; trading-specific reward shaping required
Thorp (1997)	Kelly Criterion	Portfolio theory	Optimal growth position sizing	Sensitive to parameter estimation errors; requires capping
Proposed Work (2026)	LSTM + PPO + Behavioral Profiling	NSE/BSE, 52 NIFTY 50 stocks	Integrated end-to-end pipeline; personalization; deployment	Simulated live feed; no broker execution API

The proposed system constructs a secure network tunnel by layering The design philosophy of the suggested solution platform can be described by the three key principles which define its architecture. First, modularity allows developing, testing, and updating each of the separate system elements: data processing, LSTM prediction engine, PPO decision-making module, behavioral profile, and the web-based UI, in isolation from one another. The principle of separation of concerns implies clearly assigning different functionalities to certain layers, avoiding intermingling of data handling, ML inference processes, and UI rendering. Finally, the principle of graceful degradation means that in case some of the required resources are unavailable, the system provides rule-based fallbacks instead of crashing and sending error messages to the user.

3.1 Technical Foundations

This section presents the theoretical foundations underlying the proposed stock prediction and trading framework. The primary forecasting component of the system is based on Long Short-Term Memory (LSTM) networks, which are specifically designed to model sequential and time-series data. Their ability to capture both short-term and long-term temporal dependencies makes them particularly suitable for financial market prediction.

3.1.1 Long Short-Term Memory Networks

(a) *Architecture and Gating Mechanisms*

The Long Short-Term Memory (LSTM) network, introduced by Hochreiter and Schmidhuber [hochreiter1997long](#), is an extension of the conventional Recurrent Neural Network (RNN). Unlike standard RNNs, LSTMs employ memory cells and gating mechanisms that alleviate the vanishing gradient problem, enabling the network to learn long-range temporal dependencies effectively.

An LSTM unit maintains two internal states: the cell state c_t , which serves as long-term memory, and the hidden state h_t , which represents short-term memory and is propagated to subsequent layers. Information flow within the unit is regulated through three gates: the forget gate, input gate, and output gate.

Forget Gate. The forget gate determines the amount of information from the previous cell state that should be retained:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (3.1)$$

Input Gate. The input gate controls the amount of new information that is incorporated into the memory cell:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (3.2)$$

Candidate Cell State. Candidate values for updating the memory cell are computed as:

$$\tilde{c} * t = \tanh(W_c[h * t - 1, x_t] + b_c) \quad (3.3)$$

Cell State Update. The updated cell state combines retained information from the previous state with newly acquired information:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (3.4)$$

Output Gate. The output gate determines the information from the cell state that is exposed to the hidden state:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (3.5)$$

Hidden State. The hidden state is computed as:

$$h_t = o_t \odot \tanh(c_t) \quad (3.6)$$

In Equations (3.1)–(3.6), $\sigma(\cdot)$ denotes the sigmoid activation function, $\tanh(\cdot)$ denotes

the hyperbolic tangent activation function, $\odot$ represents element-wise multiplication, W denotes trainable weight matrices, and b denotes bias vectors. The input vector at time step t is represented by x_t , while $[h_{t-1}, x_t]$ denotes the concatenation of the previous hidden state and the current input.

(b) Proposed LSTM Architecture

The proposed prediction model employs a stacked LSTM architecture to capture complex temporal patterns present in historical stock market data. Two LSTM layers, each comprising 64 hidden units, are utilized to learn hierarchical temporal representations. Fully connected layers are subsequently employed to perform regression-based stock price prediction. Dropout regularization is incorporated to improve generalization and reduce overfitting.

The architecture is summarized as follows:

- **Input Layer:** Historical sequences consisting of 30 trading days, where each day is represented by five features: Open, High, Low, Close, and Volume. All features are normalized using Min-Max scaling. ““
- **LSTM Layer 1:** Contains 64 hidden units with `batch_first=True`. A dropout rate of 0.2 is applied to the layer outputs.
- **LSTM Layer 2:** Contains 64 hidden units and produces the final hidden representation h_T .
- **Fully Connected Layer 1:** Transforms the 64-dimensional feature vector into 32 neurons using the ReLU activation function, followed by a dropout rate of 0.2.
- **Output Layer:** Maps the 32-dimensional representation to a single neuron with a linear activation function to predict the next trading day’s closing price. ““

The stacked LSTM architecture enables the model to learn hierarchical temporal representations from historical stock market data. The final output corresponds to the predicted closing price of the subsequent trading day. These predictions are subsequently utilized by the reinforcement learning agent to support portfolio allocation and trading decisions.

3.1.2 Reinforcement Learning Framework

The portfolio management and trading decision component of the proposed system is formulated as a Reinforcement Learning (RL) problem. The agent interacts with a simulated stock market environment and learns an optimal trading policy through trial-and-error interactions. Proximal Policy Optimization (PPO) is employed as the underlying RL algorithm due to its stability, sample efficiency, and strong performance in sequential decision-making tasks.

(a) *Formulation of the Problem as a Markov Decision Process*

A formal representation of the trading problem can be expressed as a Markov Decision Process (MDP) characterized by the tuple (S, A, P, R, γ) , where:

- S denotes the finite state space represented by a 34-dimensional observation vector containing stock prices, technical indicators, portfolio states, and behavioral profile features.
- A represents the discrete action space consisting of five actions:

$$A = \{HOLDBUY, HOLDSELL, BUY, SELL, IDLE\}$$

- P denotes the transition function that depends on market dynamics and stochastic changes in stock prices.
- R represents the reward function combining step-wise trading returns with the Sharpe ratio calculated over the complete trading episode.
- γ is the discount factor controlling the importance of future rewards. In the PPO implementation, $\gamma = 0.99$.

(b) *Proximal Policy Optimization*

Proximal Policy Optimization (PPO) [schulman2017proximal](#) is an on-policy reinforcement learning algorithm that trains a stochastic policy $\pi_\theta(a|s)$ by maximizing a clipped surrogate objective function. The clipping mechanism prevents excessively large policy updates and improves training stability.

$$L_{CLIP}(\theta) = E_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right] \quad (3.7)$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (3.8)$$

represents the policy update ratio, $\hat{A}_t$ is the estimated advantage function at step t , and ϵ is a hyperparameter defining the trust region size. In the proposed implementation, $\epsilon = 0.2$. The minimum operator ensures that policy updates remain within the trust region and prevents destructive updates.

The complete PPO loss function combines the clipped policy objective, value function loss, and an entropy regularization term:

$$L_{PPO}(\theta) = L_{CLIP}(\theta) - c_1 L_{VF}(\theta) + c_2 H[\pi_\theta(s_t)] \quad (3.9)$$

where $c_1 = 0.5$, $c_2 = 0.01$, L_{VF} denotes the mean squared error between predicted and target value functions, and $H[\pi_\theta]$ represents the entropy bonus encouraging policy exploration.

(c) *Reward Function*

The custom reward mechanism implemented in the TradingEnv environment aims to maximize both immediate trading profits and long-term risk-adjusted returns. The reward at each time step is defined as:

$$r_t = position_t \cdot \Delta p_t - c_{trans} - c_{risk} + \lambda \cdot \tanh(SR_{episode}) \quad (3.10)$$

where Δp_t denotes the observed price change during the current step, $c_{trans} = 0.001$ is the transaction cost coefficient, c_{risk} represents a risk penalty determined by the investor's behavioral profile, and $SR_{episode}$ is the Sharpe ratio computed over the entire episode and transformed using the hyperbolic tangent function.

The reward structure encourages profitable trading while simultaneously penalizing excessive risk-taking behavior. By incorporating both transaction costs and risk-adjusted performance measures, the agent learns trading policies that are more robust and aligned with the objectives of personalized portfolio management.

3.1.3 Kelly Criterion for Position Sizing

The Kelly Criterion is employed in the proposed framework to determine the optimal fraction of capital that should be allocated to a particular trade. Originally developed by Kelly **kelly1956new**, the criterion maximizes the long-term growth rate of capital by balancing expected returns against the probability of losses. In the context of algorithmic trading, Kelly-based position sizing provides a systematic mechanism for capital allocation based on the estimated performance of the trading strategy.

(a) *Mathematical Formulation*

According to the Kelly Criterion **thorp1997kelly**, the optimal fraction of capital f^* allocated to a trade is given by:

$$f^* = \frac{p \cdot b - q}{b} = \frac{pb - (1 - p)}{b} \quad (3.11)$$

where:

- p represents the probability of a profitable trade,
- $q = 1 - p$ denotes the probability of a losing trade,
- b represents the net odds received on the investment, corresponding to the average profit-to-loss ratio.

In the proposed trading framework, the value of p is estimated using the historical winning percentage of the PPO trading agent, while b is computed from the average profit-to-loss ratio observed during training and evaluation. The resulting Kelly fraction provides an estimate of the proportion of available capital that should be invested in the current trading opportunity.

(b) *Practical Implementation Variations*

Although the Kelly Criterion theoretically maximizes long-term portfolio growth, the use of the full Kelly fraction often results in excessive portfolio volatility and substantial drawdowns. These issues become particularly significant when the estimated probabilities and expected returns are subject to uncertainty or prediction errors **maclean2011kelly**.

To address these limitations, the proposed system employs a capped Kelly sizing

strategy that scales the Kelly fraction according to the investor's risk profile and imposes an upper investment limit:

$$f_{actual} = \min(\alpha \cdot f^*, f_{\max}) \quad (3.12)$$

where:

- $\alpha \in [0.25, 1.0]$ is a risk tolerance factor determined from the investor's behavioral risk score,
- f^* is the optimal Kelly fraction computed using Equation (3.11),
- $f_{\max} = 0.25$ represents the maximum allowable allocation, limiting any individual position to 25% of the available capital.

The risk tolerance factor enables the position sizing mechanism to adapt to different investor profiles. Conservative investors receive lower values of α , resulting in smaller position sizes, while aggressive investors are assigned higher values. The cap on maximum allocation prevents excessive concentration of capital in a single asset and reduces the risk associated with estimation errors in the Kelly parameters.

By integrating behavioral risk profiling with the Kelly Criterion, the proposed framework achieves a balance between growth optimization and practical risk management, thereby supporting more personalized and robust portfolio allocation decisions.

3.1.4 Sharpe Ratio

The Sharpe Ratio is a widely used performance metric for evaluating risk-adjusted returns in investment portfolios. Proposed by Sharpe **sharpe1966mutual**, the ratio measures the excess return generated by a portfolio relative to the amount of risk undertaken. Unlike absolute return measures, the Sharpe Ratio simultaneously considers profitability and volatility, making it particularly suitable for assessing the effectiveness of algorithmic trading strategies.

The Sharpe Ratio is defined as:

$$SR = \frac{E[R_p] - R_f}{\sigma_p} \quad (3.13)$$

where:

- $E[R_p]$ represents the expected portfolio return,
- R_f denotes the risk-free rate,
- σ_p represents the standard deviation of portfolio returns.

In the proposed framework, the risk-free rate is set to 0.06, corresponding to the approximate return of Indian government securities during the study period. The Sharpe Ratio serves as an important component of the reinforcement learning reward function and is incorporated into the PPO training process to encourage the agent to generate consistent returns while controlling portfolio risk.

The use of the Sharpe Ratio is advantageous because it rewards profitable trading behavior while simultaneously penalizing excessive volatility. Consequently, the reinforcement learning agent is encouraged to learn trading policies that achieve a balance between return maximization and risk management rather than pursuing aggressive strategies that may yield unstable profits.

In quantitative and systematic trading applications, a Sharpe Ratio greater than 0.5 is generally considered acceptable, while values above 1.0 indicate strong risk-adjusted performance. Experimental evaluation of the proposed trading framework produced a Sharpe Ratio of 0.66, indicating that the generated trading strategy achieved satisfactory risk-adjusted returns. This result demonstrates that the PPO agent was able to maintain a favorable balance between profitability and portfolio volatility throughout the evaluation period.

The incorporation of the Sharpe Ratio into both the evaluation framework and the reward design contributes to the development of a more robust trading strategy, ensuring that portfolio growth is achieved without excessive exposure to market risk.

3.2 System Architecture

The complete system is organized into four horizontal layers, as illustrated in Figure 3.1. The layered architecture provides a modular and scalable framework for integrating data acquisition, predictive modeling, reinforcement learning-based portfolio management, and user interaction components. Each layer performs a specific function

while interacting with adjacent layers to support end-to-end personalized algorithmic trading.

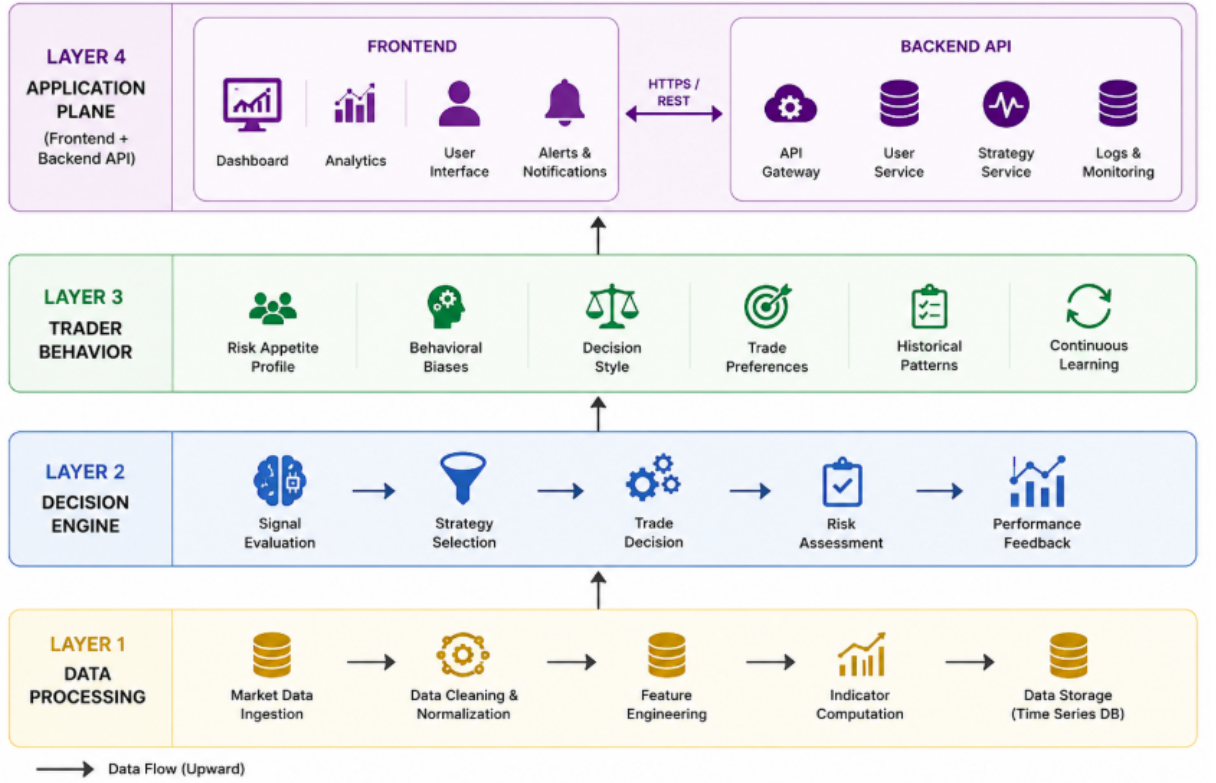


Figure 3.1: High-level architecture of the personalized algorithmic trading platform.

3.2.1 Layer 1: Data Processing

Layer 1 is responsible for market data acquisition, normalization, and feature engineering. The `normalize.symbol()` function ensures correct market-specific symbol formatting. NSE-listed stocks are appended with the `.NS` suffix, BSE-listed stocks with the `.BO` suffix, while market indices are prefixed with the caret (`^`) symbol. Historical OHLCV (Open, High, Low, Close, Volume) data are retrieved using the `yfinance` library over a specified lookback period. Both synchronous and asynchronous retrieval modes are supported, with asynchronous execution performed in a separate executor thread to prevent blocking of the FastAPI event loop.

Technical indicator computation is performed through the `compute.indicators()` function using the `pandas-ta` library. More than thirty indicators are generated and categorized as follows:

- **Trend Indicators:** SMA-20, SMA-50, SMA-200, EMA-12, EMA-26, MACD with signal line, and ADX-14.

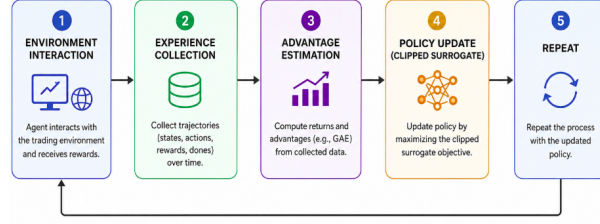


Figure 3.2: LSTM Training and Validation

- **Momentum Indicators:** RSI-14, Stochastic Oscillator (%K/%D), CCI-20, Williams %R, and ROC-10.
- **Volatility Indicators:** Upper and lower Bollinger Bands, Bollinger Bandwidth, ATR-14, and ATR expressed as a percentage of price.
- **Volume Indicators:** On-Balance Volume (OBV), Money Flow Index (MFI-14), and volume ratio relative to the 20-day average.

The `build_state()` function constructs a 30-dimensional state representation consisting of normalized price features, technical indicators, user behavioral profile embeddings, and portfolio state variables such as current position, unrealized profit and loss, and available capital ratio. This state vector is subsequently padded to 34 dimensions within the `TradingEnv` environment to satisfy the observation space requirements of the PPO agent.

3.2.2 Layer 2: Decision Engine

Layer 2 implements the reinforcement learning-based decision engine responsible for portfolio management and trade execution decisions. The custom `TradingEnv` class extends the `gymnasium.Env` interface and defines the trading environment used during agent training and evaluation.

The environment configuration includes:

- **Observation Space:** `Box(low=-∞, high=+∞, shape=(34,), dtype=float32)`
- **Action Space:** `Discrete(5)` with actions defined as:
 - 0 = HOLDBUY
 - 1 = HOLDSELL

- 2 = BUY
- 3 = SELL
- 4 = IDLE

- **Initial Capital:** 100,000 monetary units.
- **Transaction Cost:** 0.1% per trade.

The PPO agent is implemented using the Stable-Baselines3 framework with the MlpPolicy architecture. The neural network consists of two hidden layers with 256 neurons each. The learning rate is set to 3×10^{-4} and the PPO clipping parameter is fixed at 0.2.

To support personalized trading behavior, the `create_agent()` function performs risk-adaptive initialization based on the user’s behavioral profile. Conservative investors are assigned lower learning rates and higher entropy coefficients to encourage cautious exploration, whereas aggressive investors are assigned higher learning rates and lower entropy coefficients to facilitate faster convergence toward higher-risk, higher-reward trading policies.

3.2.3 Layer 3: Trader Behavior and Risk Profiling

Layer 3 is responsible for capturing and modeling individual trader behavior through a psychometric evaluation framework. The behavioral assessment consists of 30 questions distributed across five thematic dimensions: (i) capital allocation preferences, (ii) risk tolerance, (iii) overtrading tendencies, (iv) perception of market conditions, and (v) breakeven and loss-recovery preferences. Each response is mapped to a normalized value within the interval $[0, 1]$.

The responses are aggregated using domain-specific weighting techniques to generate a 15-dimensional behavioral feature vector:

$$b \in [0, 1]^{15}$$

This vector serves as a compact representation of the trader’s investment behavior and psychological characteristics.

The overall risk score is computed as the average of the behavioral vector components:

$$\rho = \frac{1}{15} \sum_{j=1}^{15} b_j \quad (3.14)$$

Based on the computed risk score, users are categorized into one of four risk profiles:

- **Conservative:** $\rho < 0.25$
- **Moderate:** $0.25 \leq \rho < 0.50$
- **Growth:** $0.50 \leq \rho < 0.75$
- **Aggressive:** $\rho \geq 0.75$

The behavioral vector b is incorporated directly into the reinforcement learning observation space, enabling the PPO agent to adapt its trading decisions according to the investor's risk preferences. Consequently, the learned trading policy becomes personalized rather than relying on a single generic strategy for all users.

Upon completion of the behavioral assessment, the system asynchronously initiates retraining of a customized PPO agent. This process allows the trading policy to be optimized according to the individual's risk profile, thereby improving personalization and investment suitability.

3.2.4 Application and API Layer

Layer 4 comprises the application and service layer responsible for user interaction, authentication, data delivery, and communication between frontend and backend components. The backend is implemented using FastAPI and exposes more than fifteen RESTful and WebSocket endpoints organized into multiple routers.

The primary API routes include:

- **/api/v1/auth:** Handles Firebase identity verification, user authentication, and JWT token generation.
- **/api/v1/trading:** Provides market data retrieval, trading signal generation, and trade logging functionality.
- **/api/v1/backtest:** Executes historical backtesting simulations and returns performance metrics.

- **/api/v1/profile:** Manages behavioral questionnaire submission, risk-score calculation, and profile updates.
- **/api/v1/ws/prices:** WebSocket endpoint used for real-time market price streaming at 30-second intervals.

The frontend application is developed using Next.js 16 and React 19. The user interface consists of nine primary pages that collectively provide authentication, portfolio monitoring, trading signal visualization, market analysis, historical backtesting, behavioral assessment, trade history management, and application configuration.

User authentication status is managed globally using Zustand with persistent localStorage support. Communication with backend services is handled through Axios-based API clients, where interceptors automatically attach Bearer authentication tokens and manage unauthorized (401) responses. This architecture ensures secure, scalable, and responsive interaction between users and the personalized algorithmic trading platform.

3.3 Deployment Modes and Resilience

The proposed trading platform supports multiple deployment configurations to accommodate different development, testing, and production requirements. These deployment modes differ in terms of storage mechanisms, authentication strategies, and database dependencies. Table 3.1 summarizes the supported deployment configurations and their intended use cases.

Table 3.1: Deployment modes and infrastructure requirements.

Mode	Storage	Authentication Mechanism	Database Required	Usage Example
DEMO	In-memory	JWT	N/A	Prototype demonstration
DEMO Cloud	Firebase Firestore	JWT + bcrypt	No	Cloud-hosted demonstration
PRODUCTION	PostgreSQL 15	JWT + bcrypt	Yes	Complete production deployment

To ensure system reliability and operational continuity, a resilient fallback mechanism has been incorporated into the deployment architecture. Whenever the trained model

files `lstm_final.pt` or `ppo_trading_final.zip` are unavailable, corrupted, or cannot be successfully deserialized during initialization, the platform automatically switches to a safe fallback mode.

In fallback mode, all trading signal requests return a structured JSON response with the following default values:

- `action = "IDLE"`
- `confidence = 0.5`
- `source = "fallback"`

This strategy prevents application failures and ensures uninterrupted API availability even when trained model artifacts are missing. The fallback response clearly indicates that predictions are not being generated by the trained models, thereby maintaining transparency for end users and external applications.

Furthermore, the platform supports automated model bootstrapping through the environment variable:

`AUTO_TRAIN_IF_MISSING=True`

When enabled, the model bootstrap module automatically initiates the training pipeline during application startup whenever the required model files are not detected. This feature simplifies deployment and recovery processes by eliminating the need for manual retraining.

The storage requirements of the machine learning models remain relatively small, facilitating efficient deployment across cloud and on-premise environments. The trained LSTM forecasting model occupies approximately 500 KB of storage, while the PPO reinforcement learning agent requires approximately 2 MB. These compact model sizes enable rapid loading, reduced memory consumption, and efficient distribution across deployment environments.

CHAPTER 4

EXPERIMENT RESULTS

4.1 Experimental Setup

4.1.1 Data Acquisition and Preprocessing

Historical OHLCV (Open, High, Low, Close, Volume) data for all 52 NIFTY 50 constituent equities were collected using the `yfinance` API over a five-year period spanning June 2019 to June 2024. The resulting dataset contained approximately 79,628 trading-day observations. Data were stored both as a consolidated `training_data.csv` file and as individual symbol-specific CSV files to facilitate efficient LSTM training and evaluation.

The collected data underwent preprocessing prior to model training. Missing values were removed, stock symbols were normalized, and all features were scaled on a per-symbol basis using the `MinMaxScaler` transformation. A sliding window approach with a sequence length of 30 trading days was employed to generate supervised learning samples for the LSTM model.

Table 4.1 summarizes the characteristics of the dataset used for model training.

Table 4.1: Dataset characteristics for LSTM training.

Attribute	Value
Stocks	52 NIFTY 50 constituents
Data source	yfinance (Yahoo Finance)
Time span	June 2019 – June 2024 (5 years)
Total rows	Approximately 79,628 OHLCV observations
LSTM training samples	23,167 (after sequence windowing)
Sequence length	30 trading days
Features per step	5 (Open, High, Low, Close, Volume)
Train/validation split	80% / 20% (chronological, per symbol)
Normalization	Per-symbol <code>MinMaxScaler</code> $\rightarrow [0, 1]$

4.1.2 LSTM Training Configuration

Training was conducted on CPU hardware consisting of an Intel Core i5-class processor with 8 GB RAM. The Adam optimizer was employed with an initial learning rate of 1×10^{-3} . To improve convergence and prevent stagnation, a ReduceLROnPlateau scheduler was utilized with a patience of five epochs and a reduction factor of 0.5.

Gradient clipping with a maximum norm of 1.0 was applied to mitigate gradient explosion during backpropagation through time. The model was trained for 30 epochs using Mean Squared Error (MSE) as the optimization objective.

4.1.3 PPO Training Configuration

The PPO agent was trained using the custom TradingEnv environment and the Stable-Baselines3 framework. Training was performed for 30,000 timesteps with hyperparameters selected based on empirical performance and stability considerations.

Table 4.2 summarizes the PPO training configuration.

Table 4.2: PPO agent training configuration.

Hyperparameter	Value
Algorithm	PPO (Stable-Baselines3)
Policy	MlpPolicy, net_arch=[256,256]
Learning rate	3×10^{-4}
Clip range (ϵ)	0.2
Discount factor (γ)	0.99
GAE lambda	0.95
Batch size	64
Number of epochs	10
Training timesteps	30,000
Evaluation episodes	5
Initial capital	100,000 units
Transaction cost	0.1% per trade

4.2 LSTM Performance Evaluation

4.2.1 Quantitative Results

The quantitative performance metrics obtained during training and evaluation are summarized in Table 4.3.

Table 4.3: LSTM model training and evaluation metrics.

Metric	Value
Architecture	2-layer LSTM, hidden=64, seq len=30, features=5
Total parameters	Approximately 39,000
Training samples	23,167
Best validation loss (MSE)	0.000180 (Epoch 21)
Final validation loss (MSE)	0.000230
Optimizer	Adam ($\eta = 0.001$)
Scheduler	ReduceLROnPlateau (patience=5, factor=0.5)
Gradient clipping	max norm=1.0
Training time (CPU)	Approximately 5 minutes

4.2.2 Interpretation

The best validation MSE of 0.000180 represents the normalized prediction error on MinMax-scaled stock price data. Since all target values and predictions are constrained to the interval $[0, 1]$, the resulting error values are inherently small. Nevertheless, comparison with randomly initialized baselines, which typically yield MSE values between 0.08 and 0.15, demonstrates that the model successfully learned meaningful predictive patterns from the historical market data.

The relatively small increase from the best validation loss (0.000180) to the final validation loss (0.000230) suggests only minor overfitting toward the end of training. This behavior is expected given the pooled multi-stock training strategy, where stock-specific patterns are partially averaged across the combined dataset.

The inference module converts predicted price movements into trading signals according to predefined thresholds:

- Predicted change $> +1\%$: BUY signal with confidence scaled between 0.5 and 0.95.
- Predicted change $< -1\%$: SELL signal with confidence scaled inversely according to the magnitude of the predicted decline.
- Otherwise: IDLE signal with confidence fixed at 0.5.

This threshold-based mechanism provides a simple yet effective mapping between continuous price predictions and discrete trading decisions used by the reinforcement learning framework.

4.3 Evaluation of PPO Agent Performance

4.3.1 Performance in terms of Return and Risk-Adjusted Metrics

The performance of the PPO agent was evaluated using five independent test episodes within the custom trading environment. Evaluation focused on both absolute returns and risk-adjusted performance measures to assess the effectiveness of the learned trading policy. Table 4.4 summarizes the key performance metrics obtained during evaluation.

Table 4.4: PPO agent performance metrics across five test episodes.

Metric	Value
Average return	132.28%
Sharpe ratio	0.66
Evaluation episodes	5
Training timesteps	30,000
Action space	Discrete(5): HOLDBUY, HOLDSELL, BUY, SELL, IDLE
Observation space	Box(34,) continuous
Network	[256, 256] MLP
Training time (CPU)	Approximately 2–5 minutes

4.3.2 Results Interpretation

The PPO agent achieved an average return of 132.28% across five evaluation episodes, indicating strong performance within the simulated trading environment. While this result demonstrates the effectiveness of the learned policy, direct comparison with results reported in the literature remains challenging due to differences in datasets, market conditions, evaluation periods, transaction cost assumptions, and training methodologies.

(a) *Sharpe Ratio Importance*

The Sharpe ratio of 0.66 was computed using episode-level returns and a risk-free annual return of 6%. In quantitative trading, Sharpe ratios exceeding 0.5 are generally considered acceptable, whereas values greater than 1.0 indicate strong risk-adjusted performance. The obtained Sharpe ratio of 0.66 suggests that the PPO agent successfully generated returns that adequately compensated for the level of risk undertaken. This outcome demonstrates the effectiveness of incorporating the Sharpe ratio into the reward function, enabling the agent to balance profitability and risk management during policy optimization.

(b) Return Value Importance

The reported average return of 132.28% represents cumulative performance obtained across multiple historical evaluation episodes. The duration of these episodes may span several months of historical market activity depending on the selected evaluation window. Consequently, the reported returns should not be interpreted as guarantees of future live-trading performance. Real-world trading environments introduce additional challenges such as market slippage, liquidity constraints, latency, and data quality issues that may reduce realized returns relative to simulation results.

(c) Training Performance

The PPO training process required approximately 30,000 timesteps and completed within two to five minutes on standard CPU hardware. This demonstrates that the proposed configuration is computationally efficient and suitable for academic research, rapid prototyping, and small-scale deployment scenarios. The neural network architecture consisting of two hidden layers with 256 neurons each contains approximately 132,000 trainable parameters, providing sufficient representational capacity to learn effective trading policies while maintaining manageable training times.

4.4 System Performance**4.4.1 Signal Generation Latency**

The end-to-end signal generation pipeline was evaluated to determine the responsiveness of the proposed trading platform. The measured latency includes market data retrieval, technical indicator computation, LSTM inference, PPO-based decision making, and API response serialization.

Experimental results indicate an average processing time of approximately 500 milliseconds per request under normal operating conditions. This level of latency is sufficiently low for the intended use case of daily and medium-frequency trading signal generation. The achieved response time enables users to obtain trading recommendations with minimal delay while maintaining the computational complexity required by the forecasting and reinforcement learning components.

Although the current architecture performs adequately for daily trading scenarios, it is not optimized for high-frequency or intraday trading applications. Such environments would require specialized infrastructure, including low-latency market feeds, event-driven

processing architectures, and optimized model inference pipelines to meet stricter timing constraints.

4.4.2 WebSocket Data Pushing

Real-time market updates are delivered through the WebSocket endpoint `/api/v1/ws/prices`. The service broadcasts market price information to subscribed clients at fixed intervals of 30 seconds. While this implementation relies on periodic polling rather than true event-driven streaming, it is sufficient for educational, research, and medium-frequency trading applications.

Whenever updated market information becomes available, the server distributes the data to all clients subscribed to the corresponding stock symbol. Client connections are managed through a set of WebSocket event handlers that support subscription management and connection maintenance.

The implemented event handlers include:

- **subscribe**: Registers a client for updates related to a specific stock symbol.
- **unsubscribe**: Removes a client from the subscription list of a particular symbol.
- **set**: Updates client-specific configuration parameters.
- **ping**: Maintains active connections and verifies client availability.

This architecture provides an efficient mechanism for delivering market information to multiple users while maintaining low server overhead and acceptable responsiveness.

4.4.3 Trader Behavior Profiler

The trader behavior profiler constitutes a key component of the personalization framework. It enables the system to adapt trading recommendations according to individual risk preferences and behavioral characteristics.

The RiskProfiler module utilizes a psychometric questionnaire consisting of 30 questions designed to evaluate investor behavior across multiple dimensions, including risk tolerance, capital allocation preferences, trading habits, and market perception. User responses are transformed into a 15-dimensional behavioral feature vector:

$$b \in [0, 1]^{15}$$

The behavioral vector is subsequently aggregated into a scalar risk score:

$$\rho \in [0, 1]$$

which represents the overall risk appetite of the investor.

The resulting risk profile directly influences portfolio management decisions. Personalized position sizing is performed using the modified Kelly Criterion formulation presented in Equation (3.11), where the risk adjustment factor α is determined according to the user’s behavioral characteristics. Conservative users are assigned lower allocation factors, whereas aggressive users receive higher allocation factors, enabling the system to tailor investment decisions to individual preferences.

Following completion of the behavioral assessment, the generated behavioral vector is incorporated into the reinforcement learning observation space. The PPO agent is then retrained asynchronously using the updated user profile, allowing the trading policy to adapt dynamically to the investor’s unique behavioral traits. This personalization mechanism distinguishes the proposed framework from conventional algorithmic trading systems that apply identical trading strategies to all users regardless of their risk preferences.

4.5 Reliability and Scalability Discussion

4.5.1 Fallback Mechanism

The proposed system incorporates a resilience architecture designed to maintain service availability even when trained machine learning models are unavailable. This mechanism ensures that the failure, corruption, or absence of model artifacts does not result in service interruption or degraded user experience.

When the required LSTM or PPO model files cannot be loaded successfully, the signal generation endpoints automatically switch to a fallback mode. Instead of returning an error response, the system generates a valid JSON response conforming to the `SignalResponse` schema. An additional field, `is_fallback=True`, is included in the response to indicate that the prediction was generated through the fallback mechanism rather than the trained models.

This design allows the frontend application to display appropriate notifications to users while preserving full application functionality. The fallback mechanism is particularly

valuable during fresh deployments, system recovery procedures, or model retraining phases where trained artifacts may not yet be available. As a result, the platform remains operational and responsive under a wide range of failure scenarios.

4.5.2 Multi-Stock Scalability

The current implementation has been evaluated using all 52 constituent stocks of the NIFTY 50 index. The architecture was intentionally designed to be stock-agnostic, enabling support for additional equities without requiring modifications to the core system components.

Expansion of the supported stock universe requires only the addition of new symbols to the stock universe configuration. The data acquisition pipeline, feature engineering modules, LSTM forecasting component, and PPO-based trading environment automatically adapt to newly introduced symbols without architectural changes.

The modular design provides sufficient scalability for medium-sized stock universes. However, scaling to several hundred or thousands of equities would require additional optimization strategies. Potential enhancements include:

- Caching of technical indicator computations to reduce redundant calculations.
- Parallel processing of market data and feature generation tasks.
- Distributed model inference for simultaneous signal generation across multiple symbols.
- Load balancing and horizontal scaling of API services.
- Dedicated message queues and asynchronous processing pipelines.

These improvements would enable the platform to maintain acceptable response times and computational efficiency as the number of supported securities increases.

4.6 Limitations of Evaluation

Although the experimental results demonstrate the effectiveness of the proposed framework, several limitations should be acknowledged when interpreting the reported findings.

4.6.1 Simulation Fidelity

The evaluation was conducted using a historical backtesting environment that simplifies several aspects of real-world trading. Factors such as market impact, order book dynamics, slippage, latency, partial order fills, and liquidity constraints were not explicitly modeled. Consequently, actual trading performance in live market conditions may differ from the results obtained in simulation.

4.6.2 Overfitting Risk

Both the LSTM forecasting model and the PPO trading agent were trained and evaluated using data drawn from the same five-year period and the same set of 52 NIFTY 50 stocks. Although separate training and validation partitions were employed, the evaluation does not fully assess the ability of the models to generalize to unseen market environments or securities outside the training universe.

Future work should investigate performance under out-of-distribution conditions, including different stock universes, market sectors, and economic regimes not represented within the current dataset.

4.6.3 Hardcoded PPO Confidence

The confidence values reported by the PPO agent are currently represented using a fixed confidence score of 0.8 rather than the true action probabilities produced by the policy network. While this simplification provides a consistent confidence measure for signal presentation, it does not accurately reflect the agent’s underlying uncertainty.

Extracting and exposing the actual softmax probabilities from the policy network would improve interpretability and provide users with more meaningful confidence estimates for trading recommendations.

4.6.4 State Dimension Padding

The `build_state()` function generates a 30-dimensional feature vector that is subsequently padded to 34 dimensions within the `TradingEnv` environment. Although this approach satisfies the observation space requirements of the PPO agent, it introduces redundant zero-valued features into the state representation.

While the impact on performance is minimal, the PPO network must implicitly learn to ignore these padded dimensions. A more efficient implementation would directly generate

a feature vector matching the required observation space size.

4.6.5 Absence of Walk-Forward Validation

The current evaluation methodology does not incorporate walk-forward validation, Monte Carlo simulation, or benchmarking against traditional investment strategies such as buy-and-hold. Consequently, the statistical robustness of the reported performance metrics remains limited.

Future evaluations should include rolling-window validation, benchmark comparisons, sensitivity analyses, and stress-testing under different market conditions. Such experiments would provide stronger evidence regarding the generalizability and practical effectiveness of the proposed trading framework.

CHAPTER 5

CONCLUSION

5.1 Conclusion

This report presented the design, implementation, and evaluation of a personalized algorithmic trading platform that integrates deep learning, reinforcement learning, and behavioral profiling to support intelligent investment decision-making. The proposed system was specifically developed for the Indian equity market and utilizes historical data from 52 constituent stocks of the NIFTY 50 index listed on the National Stock Exchange (NSE) and Bombay Stock Exchange (BSE).

The architecture of the platform is organized into three primary layers. The first layer performs market data acquisition, preprocessing, and technical indicator generation. The second layer employs reinforcement learning techniques to generate trading decisions and portfolio management actions. The third layer captures investor behavior through psychometric profiling and incorporates user-specific risk preferences into the trading process. These components are integrated through a FastAPI backend and a Next.js-based frontend, providing a complete end-to-end trading support platform.

Experimental evaluation demonstrated the effectiveness of the proposed framework. The LSTM-based forecasting model was trained using 23,167 samples generated from five years of historical market data and achieved a best validation Mean Squared Error (MSE) of 0.000180. This result indicates that the model successfully captured temporal dependencies within stock price sequences and generated highly accurate next-day price predictions.

The reinforcement learning component, implemented using Proximal Policy Optimization (PPO), successfully learned trading policies that balanced profitability and risk management. After training for 30,000 timesteps, the PPO agent achieved an average portfolio return of 132.28% and a Sharpe ratio of 0.66 across evaluation episodes. The obtained Sharpe ratio confirms that the agent generated favorable

risk-adjusted returns and effectively utilized the reward structure designed around portfolio performance and risk control.

The behavioral profiling subsystem further enhanced the personalization capabilities of the platform. By transforming responses from structured psychometric questionnaires into a 15-dimensional behavioral vector and corresponding risk score, the system was able to customize position sizing through a modified Kelly Criterion and adapt reinforcement learning policies to individual investor preferences.

From a deployment perspective, the platform supports multiple operational modes and incorporates a resilient fallback mechanism that maintains service availability even when trained model artifacts are unavailable. The modular design, web-based architecture, and support for both REST and WebSocket communication make the platform suitable for educational, research, and prototype deployment scenarios.

Overall, the results demonstrate that integrating deep learning, reinforcement learning, and behavioral profiling within a unified framework can provide a practical and personalized approach to algorithmic trading. The proposed platform represents a significant step toward intelligent investment assistance systems capable of adapting to both market conditions and individual investor characteristics.

5.2 Future Work

Several opportunities exist for extending and improving the proposed platform in future research and development efforts.

One important enhancement would be the integration of real-time market data feeds obtained directly from regulated exchange data providers. Replacing the current polling-based `yfinance` infrastructure with low-latency streaming data would improve data freshness and enable more responsive trading decisions.

Another significant extension involves integration with brokerage platforms such as Zerodha Kite Connect, Upstox API, and Angel One SmartAPI. Such integration would allow automated trade execution, transforming the system from a decision-support platform into a fully automated algorithmic trading solution.

Future versions of the platform could also incorporate experiment tracking and model management frameworks such as MLflow or Weights & Biases. These tools would facilitate model versioning, performance monitoring, drift detection, automated

retraining, and reproducibility of experimental results, thereby improving the platform’s production readiness.

The scope of supported financial instruments may be expanded beyond equities to include commodities, foreign exchange markets, exchange-traded funds (ETFs), and cryptocurrencies. Supporting multiple asset classes would increase the practical applicability of the framework and enable more diversified portfolio management strategies.

From a machine learning perspective, future work could investigate advanced forecasting architectures such as Transformers, Temporal Fusion Transformers (TFT), Informer networks, and hybrid LSTM-Transformer models. Similarly, reinforcement learning performance may be improved through the adoption of more sophisticated algorithms such as Soft Actor-Critic (SAC), Twin Delayed Deep Deterministic Policy Gradient (TD3), or multi-agent reinforcement learning frameworks.

Finally, more rigorous evaluation methodologies should be incorporated, including walk-forward validation, benchmark comparisons against buy-and-hold strategies, Monte Carlo simulations, and testing across different market regimes. These improvements would provide stronger evidence regarding the robustness, generalizability, and real-world effectiveness of the proposed personalized algorithmic trading platform.